

Advanced Lesson 7 - Exercises: Internal Form, Boolean Tests, Attributes

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: Head and FullForm of E^x and x^E

```
Clear[x];  
{Head[ex], FullForm[ex]}  
  
{Head[xe], FullForm[xe]}
```

Both have Head Power, and their full forms are Power[E, x] and Power[x, E].
Exponentiation is not a special construct - it is an ordinary function application, and the superscript is only a display convention. The two expressions differ solely in the order of the arguments, which is the whole difference between an exponential and a power function.

2: Head, FullForm and TreeForm of $x + y^2 \{u, v, w\}$

```
Clear[x, y, u, v, w];  
x + y2 {u, v, w}  
  
Head[x + y2 {u, v, w}]  
  
FullForm[x + y2 {u, v, w}]  
  
TreeForm[x + y2 {u, v, w}]
```

The expression evaluates to $\{x + u y^2, x + v y^2, x + w y^2\}$, and its Head is List, not Plus. That is the surprise, and it is worth being precise about

why. Plus and Times both carry the Listable attribute, so when either meets a list it threads elementwise. The scalar x is added to each of the three products, and the list structure is pulled to the outside. The evaluated expression is a list of three sums, not a sum involving a list.

TreeForm draws the same fact: List at the root, three Plus nodes below it, and Times and Power below those.

3: Is Pi an integer, a rational, a real?

`Head[ $\pi$ ]`

`NumericQ[ $\pi$ ]`

`NumberQ[ $\pi$ ]`

`Positive[ $\pi$ ]`

Head[Pi] returns Symbol. Pi is not stored as any kind of number at all - it is a symbol carrying the attribute Constant and a set of built-in simplification rules. There is no numeric representation of it anywhere until N is applied.

NumericQ[Pi] returns True: it asks whether the expression could be evaluated to a number, and Pi certainly could.

NumberQ[Pi] returns False: it asks whether the expression already is an explicit number, and a symbol is not.

Positive[Pi] returns True - Mathematica knows the sign of this particular constant without approximating it.

4: Is "This is a string" a list or a string?

`ListQ["This is a string"]`

`StringQ["This is a string"]`

False, then True. A string is an atom with Head String; it is not a list of characters, and Length of it is 0. Converting one into an actual list of characters takes Characters, which is Lesson 14's business.

5: Is 3 a member of {1, 2, 3, 4, 5}?

`MemberQ[{1, 2, 3, 4, 5}, 3]`

True. MemberQ tests at level 1 only, and its second argument is a pattern, not merely a value - MemberQ[{1, 2, 3}, _Integer] is also True.

6: A sign function built from Boolean tests

```
Clear[f, x];
f[x_] := +1 /; Positive[x];
f[0] = 0;
f[x_] := -1 /; Negative[x]

??f
```

"??f" lists the stored definitions in the order Mathematica will try them, top down. It puts `f[0]` first, because a literal argument is more specific than a pattern, and it keeps the two conditional pattern definitions in the order they were entered - it can rank a literal against a pattern, but it has no way to rank two conditions against each other, so among those it preserves entry order. That does not matter here, because `Positive` and `Negative` are mutually exclusive; it matters a great deal when the conditions overlap, which is the subject of Lesson 9's Exercise 3.

Note that the zero case deliberately uses a literal definition rather than a Boolean test, as the exercise requires.

```
{f[-2], f[0.023`], f[0], f[0.`], f[ $\pi$ ], f[3 + 4 I], f[a]}
```

Reading the results. `f[-2]` is -1 and `f[0.023]` is +1, as expected. `f[0]` is 0 by the literal rule. `f[Pi]` is +1, because `Positive` knows the sign of `Pi`.

`f[3 + 4 I]` returns unevaluated: a complex number is neither positive nor negative, so no condition succeeds and no rule applies. `f[a]` likewise returns unevaluated, because the sign of an unassigned symbol is unknown - and returning unevaluated is the correct answer, not a failure.

`f[0.0]` is the one that repays attention: it also returns unevaluated. The literal definition was written for the exact integer 0, and 0.0 is a `Real` - a different expression, which the literal rule does not match. Neither conditional rule fires either, because `Positive[0.]` and `Negative[0.]` are both `False`. So the machine zero falls through every case. This is the standard hazard of covering a domain with a literal for the boundary point: the boundary has more than one representation, and the literal only catches one of them. Writing the zero case as a condition on the value rather than as a literal would close the gap.

```
Plot[f[x], {x, -2, 2}]
```

The plot is a step: -1 on the left, +1 on the right, with the jump at the origin drawn as a near-vertical connecting segment because `Plot` joins its sample points.

7: Attributes, and why $x = 2$ hurts Integrate but not Plot

```
Attributes[Plot]
```

```
Attributes[Integrate]
```

```
Clear[x];
```

```
x = 2;
```

```
Plot[x2, {x, 0, 1}]
```

```
Plot[ $\int \sin[x] \, dx$ , {x, 0, 1}]
```

What actually happens, and one correction to the exercise's premise. In this version of the language `Attributes[Plot]` returns `{Protected, ReadProtected}` - `HoldAll` is no longer listed, though `Plot` still holds its first argument and still localises the plot variable internally. The behaviour the exercise describes is intact; the documented attribute that used to explain it is not. The first `Plot` works because of that holding and localisation: the outer assignment $x = 2$ is invisible inside the plot, where x is bound to each sample point in turn. The graphic is the parabola on $[0, 1]$, exactly as if no assignment had been made.

The second `Plot` fails, and the messages say why: `Integrate::ilim, "Invalid integration variable or limit(s) in 0.0000204286"` and so on, one for each sample point. Because `Plot` holds its first argument, the integral is not computed once symbolically - it is re-evaluated at every sample point, with x bound to a number. `Integrate` has no holding attributes, so it receives `Integrate[Sin[0.0000204286], 0.0000204286]` and refuses.

```
Clear[x];
```

```
Plot[Evaluate[ $\int \sin[x] \, dx$ ], {x, 0, 1}]
```

What must be done, and why. Wrap the first argument in `Evaluate`. That forces the integral to be evaluated once, symbolically, before `Plot` ever begins sampling, so `Plot` receives `-Cos[x]` and samples that. The reason it must be done is precisely the holding: without `Evaluate`, `Plot` passes the unevaluated integrand through to every sample point, and an integral whose variable has been replaced by a number is meaningless.

As for why $x = 2$ had no effect on `Plot` but did on `Integrate`: `Plot` holds and localises its variable, so it never consults the global value; `Integrate` does neither, so it receives whatever the global assignment has already substituted.

8: Table and Integrate together

```
Attributes[Table]
```

```
Attributes[Integrate]
```

```
Clear[x, n];
```

```
x = 2;
```

```
n = 3;
```

```
Table[ $\int \sin[nx] dx$ , {n, 0, 10}]
```

Parts 1 and 2. `Attributes[Table]` returns `{HoldAll, Protected}`; `Table` really does hold its arguments, and it is documented as doing so. `Integrate` holds nothing.

Part 2's evaluation fails with `Integrate::ilim` again. `Table` holds its first argument and localises `n`, so the assignment `n = 3` is harmless - `Table` overrides it with 0, 1, 2, ..., 10 in turn. But nothing localises `x`, so the global `x = 2` substitutes straight into the integrand and the integration variable becomes the number 2.

```
Clear[x];
```

```
n = 3;
```

```
Table[ $\int \sin[nx] dx$ , {n, 0, 10}]
```

Part 3, making it work. Clearing `x` is enough. The table then returns `{0, -Cos[x], -Cos[2x]/2, -Cos[3x]/3, ..., -Cos[10x]/10}`, and note that `n = 3` is still assigned and still does no damage, because `Table`'s iterator shadows it.

Part 4. The integrals were evaluated inside the table's iteration, once per value of `n`, after `Table` had substituted that value into the held body. They were not evaluated before `Table` ran, precisely because `HoldAll` prevented it.

Part 5. Each integral was evaluated with whatever `x` meant at that moment. With `x` cleared, `x` is a free symbol and the integration is symbolic. With `x = 2`, `Integrate`'s lack of holding attributes means the substitution had already happened and the call was malformed.

Part 6. The assignment `x = 2` had no effect on `Table` because `Table` holds its arguments and never looks up `x` itself; it had an effect on `Integrate` because `Integrate` evaluates its arguments normally, and by then the substitution was already done.

Part 7. The assignment `n = 3` had no effect on `Table` because `Table`'s iterator specification introduces `n` as a local variable, in the same way a bound variable in mathematics shadows a free one of the same name.

9: Locating subexpressions with Position

```
Clear[x, y, u, v];
```

```
x + y2 {u, v}
```

```
TreeForm[x + y2 {u, v}]
```

Parts 1 and 2. The expression evaluates to $\{x + u y^2, x + v y^2\}$ - a list of two sums, for the listability reason given in Exercise 2. TreeForm shows List at the root with two Plus children, each of which has x and a Times node, and each Times node has a symbol and a Power node.

```
Position[x + y2 {u, v}, x + u y2]
```

```
Position[x + y2 {u, v}, u y2]
```

```
Position[x + y2 {u, v}, y2]
```

```
Position[x + y2 {u, v}, u]
```

```
Position[x + y2 {u, v}, y]
```

Parts 3 to 7. The positions are $\{\{1\}\}$, $\{\{1, 2\}\}$, $\{\{1, 2, 2\}, \{2, 2, 2\}\}$, $\{\{1, 2, 1\}\}$ and $\{\{1, 2, 2, 1\}, \{2, 2, 2, 1\}\}$ respectively.

Read them as paths down the tree. Position 1 is the first summand.

Position $\{1, 2\}$ is its second part, the Times node. Position $\{1, 2, 1\}$ is u,

and $\{1, 2, 2\}$ is y^2 . The last two searches return two positions each,

because y^2 occurs in both summands and so does y inside it. Nothing here is special to arithmetic - Position walks any expression the same way.

```
Clear[x, y, u, v];
```

```
TableForm[Table[TreeForm[(x + y2 {u, v})[[i]], {i, 1, 2}]]]
```

```
TableForm[Table[TreeForm[(x + y2 {u, v})[[i, j]], {i, 1, 2}, {j, 1, 2}]]]
```

Part 8. The first table pulls out the two summands and draws each as a tree;

the second descends one level further and draws the four second-level parts.

The indices used are exactly the paths Position returned.

```
Clear[x, y, u, v];
```

```
TreeForm[{x + u y2, x + v y2}]
```

```
{x + u y2, x + v y2}[[0]]
```

```
{x + u y2, x + v y2}[[1, 0]]
```

```
{x + u y2, x + v y2}[[1, 2, 0]]
```

```
{x + u y2, x + v y2}[[1, 2, 2, 0]]
```

Part 9. Part index 0 extracts the head rather than an argument. So $[[0]]$

gives List, $[[1, 0]]$ gives Plus (the head of the first summand), $[[1, 2, 0]]$

gives Times, and `[[1, 2, 2, 0]]` gives Power. Walking the zero index down the same path recovers exactly the chain of heads that TreeForm draws as internal nodes - which is the point the whole lesson has been building towards: every expression is a tree, and Part addresses it.